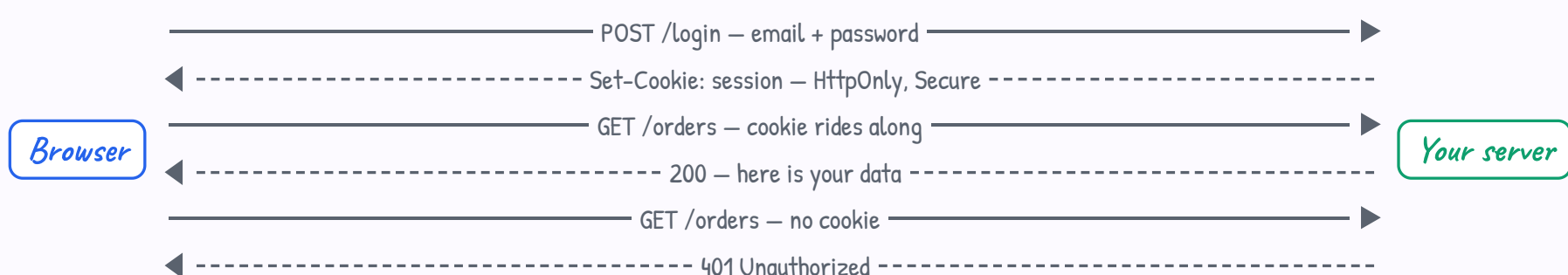




Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks



every scheme in this book is a different answer to one question: how does request #2 know it is you?

What's inside

- ✓ AuthN vs AuthZ — not the same word
- ✓ Storing passwords — hashing and salting
- ✓ Sessions — the server remembers
- ✓ JWT — signed, not encrypted
- ✓ Sessions vs JWT — the real trade-off
- ✓ OAuth 2.0 — not a login system
- ✓ OpenID Connect and SSO
- ✓ 2FA and TOTP
- ✓ Refresh tokens and rotation
- ✓ Service to service — API keys, mTLS
- ✓ Interview questions

Who it's for

- ✓ Developers who copied an auth tutorial and shipped it
- ✓ Backend engineers preparing for interviews
- ✓ Anyone who has put data inside a JWT
- ✓ Anyone who cannot explain why OAuth is not login

Why it matters

- ✓ Every flow drawn as actual traffic
- ✓ The mistakes named, not just the happy path
- ✓ Plain language before any jargon
- ✓ Ends with interview answers you can say out loud

Comment AUTH and I will DM you the full PDF



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works – and where it actually breaks

1

AuthN vs AuthZ

Two different questions, one confusing abbreviation



Authentication – who are you?

Proving **identity**. It happens once, at the door.

- Something you **know** – a password
- Something you **have** – a phone, a security key
- Something you **are** – a fingerprint, a face
- Answer: "this request belongs to user 91"



Authorization – what may you do?

Checking **permission**. It happens on every single request after that.

- User 91 may read order 4021
- User 91 may **not** read order 4022
- Roles, scopes, ownership checks
- Answer: "allowed" or **403 Forbidden**

1

401 vs 403



- **401 Unauthorized** – I don't know **who** you are
- **403 Forbidden** – I know who you are, and **no**
- The names are backwards; the meanings are not

2

The classic bug



- Logging a user in and then **never checking ownership**
- **/orders/4022** returns someone else's order
- Authenticated is not the same as **entitled**

3

Where each lives



- AuthN at the edge – once, at the gateway
- AuthZ **next to the data** – every service, every query

Key

Takeaways

- ✓ AuthN = who you are, once. AuthZ = what you may do, **every request**
- ✓ **401** means unknown; **403** means known and refused
- ✓ Most real breaches are **missing authorization**, not broken authentication



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks

2

Storing passwords

The one place where slow is the feature



Never store the password

You store a **hash** — a one-way fingerprint. At login you hash what was typed and compare fingerprints.

- Plain text: one leak and every account is gone
- Encryption is **wrong** too — encryption can be reversed
- Hashing cannot be reversed, only guessed at



Salt — per user, not secret

- A random value stored **beside** the hash
- Two users with the same password get **different** hashes
- Kills precomputed **rainbow tables** outright
- It does not need to be hidden — it needs to be unique

1

Why SHA-256 is wrong



- It is built to be **fast** — billions of guesses per second
- Fast is exactly what an attacker wants
- It is the right tool for integrity, the wrong one for passwords

2

Use a slow hash



- **bcrypt**, **scrypt**, **Argon2** — deliberately expensive
- A tunable **work factor** you raise as hardware improves
- ~100 ms per login is fine; it makes bulk guessing hopeless

3

Pepper



- An app-wide secret mixed in, kept **outside** the database
- A stolen DB alone is then not enough
- Optional — the slow hash matters far more

4

Compare in constant time



- A comparison that exits early **leaks** how much matched
- Use the library's verify function, never

==

Key

- ✓ Hash, never encrypt; **salt per user**; store salt alongside

Takeaways

- ✓ Use bcrypt / scrypt / Argon2 — **slow on purpose** — never SHA or MD5
- ✓ Raise the work factor as machines get faster; it is a dial, not a constant



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works – and where it actually breaks

3

Sessions – the server remembers

The oldest answer, and still a very good one



How it works

On login the server creates a session record and hands back an **opaque id** in a cookie. The id means nothing on its own.

- Server stores: session id → user id, expiry, device
- Browser stores: just the id, and sends it automatically
- Every request: look the id up, load the user
- Logout = **delete the row**. Instant, everywhere



The cookie flags that matter

- **HttpOnly** – JavaScript cannot read it, so XSS cannot steal it
- **Secure** – HTTPS only, never sent in clear
- **SameSite** – not attached to other sites' requests, blunting CSRF
- Missing any one of these is the actual vulnerability

1

The cost



- A lookup on **every request** – usually Redis, so microseconds
- State to store and expire
- This is a real cost, and it is usually small

2

Scaling



- Sessions in one server's memory **pin** users to that server
- Put them in shared storage and any server can serve anyone

3

What you get back



- **Instant revocation** – the thing tokens struggle with
- Full visibility: list devices, kill one, kill all

Key

- ✓ An opaque id in a cookie; the meaning lives **server-side**

Takeaways

- ✓ **HttpOnly** + **Secure** + **SameSite** – all three, or you have a hole
- ✓ Costs a lookup per request; buys **revocation that actually works**



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works – and where it actually breaks

4

JWT – signed, not encrypted

The most used and most misunderstood token

What a JWT actually is

header.payload.signature

three base64url parts joined by dots – the first two are plain text to anyone holding the token

🖋 Signed means tamper-evident

The signature proves the payload has not been changed. It does nothing to hide it.

- Anyone can paste the token and read the payload
- Change one character and the signature stops matching
- So: never put anything secret inside a JWT
- It carries claims – sub, exp, role – not secrets

⚡ Stateless is the point

- The server verifies the signature and trusts the claims
- No database lookup – that is the whole appeal
- Any service holding the public key can verify it
- Ideal across service boundaries

Key

- ✓ Three base64url parts; signed for integrity, readable by anyone

Takeaways

- ✓ No lookup needed – which is the benefit and, next page, the problem
- ✓ Put claims in it, never secrets



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks

5

Sessions vs JWT

The trade-off is revocation, and nothing else



Where the truth lives

	Session	JWT
Where the truth lives	server	inside the token
Lookup per request	yes	no
Revoke right now	delete the row	you cannot
Token contents	opaque id	readable claims
Across services	shared store needed	public key is enough
Size on the wire	tiny	hundreds of bytes, every request

1

You cannot log out a JWT



- It stays valid until `exp`, whatever you do
- Ban a user and their token **still works** until it expires
- Every real fix reintroduces a lookup — a **deny list**

2

So keep it short



- Access token: **minutes**, not days
- Then a refresh token buys a new one
- Short expiry is the revocation story

3

Honest default



- One web app, one company? **Sessions** — simpler and revocable
- Many services or third parties? Short JWT + refresh

4

The anti-pattern



- A long-lived JWT in `localStorage`
- Readable by any XSS, and **unrevocable** for its whole life

Key

- ✓ The only real difference: a session can be **killed**, a JWT expires

Takeaways

- ✓ Stateless buys scale and costs control — choose knowingly
- ✓ Default to sessions for one app; JWT when **many parties must verify**



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

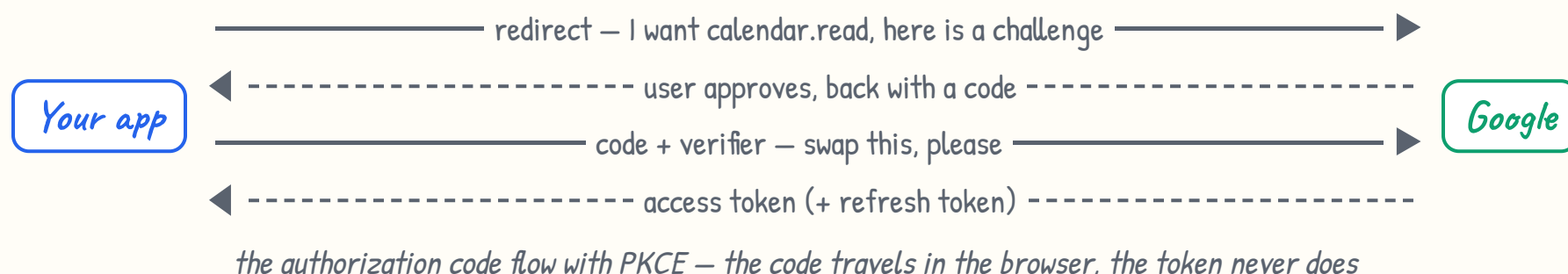
How login actually works — and where it actually breaks

6

OAuth 2.0 — not a login system

The most common misunderstanding in this book

Authorization Code + PKCE



It is about delegation

OAuth lets your app act on a user's behalf without ever seeing their password. That is authorization, not identity.

- "Let this app read my calendar" — a **scope**
- The token says what is permitted, not who you are
- Deducing identity from it is where apps go wrong



Why the two-step dance

- The **code** is useless alone and dies in seconds
- It is exchanged **server-to-server** for the real token
- **PKCE** ties the exchange to whoever started it
- So a stolen code cannot be redeemed by anyone else

Key

- ✓ OAuth answers "may this app do X for this user" — never "who is this user"

Takeaways

- ✓ Authorization Code + PKCE is the default flow for web, SPA and mobile alike
- ✓ The short-lived code exists so the **token never touches the browser URL**



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks

7

OpenID Connect and SSO

The layer that actually does login



OIDC = OAuth + identity

A thin standard layer on top of OAuth 2.0 that adds one thing: an **ID token** — a JWT describing the user.

- Ask for the **openid** scope and you get it back
- It carries **sub**, **email**, **name**, and who issued it
- The access token is for **APIs**; the ID token is for **you**
- "Sign in with Google" is OIDC, not plain OAuth



SSO — one login, many apps

- A single **identity provider** holds the session
- Each app redirects there and gets a token back
- New app? No new passwords, no new user table
- Offboard once and **every** app closes

Key

- ✓ OAuth alone has **no concept of a user** — OIDC adds the ID token

Takeaways

- ✓ Access token → call APIs. ID token → know who signed in
- ✓ SSO centralises the session: **one place to grant, one place to revoke**



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks

8

2FA and TOTP

Where the 6 digits come from



No code is ever sent

At setup, server and phone share one **secret** — that is what the QR code is. After that they never talk.

- Both hash the **secret + the current 30-second window**
- Both get the same six digits, independently
- The server accepts the neighbouring window for clock drift
- Offline, on a plane, still works



Strength order

- **Security key / passkey** — phishing-resistant, strongest
- **TOTP app** — strong, and free
- Push approval — strong until people tap out of habit
- **SMS** — weakest: SIM swap, interception

1

TOTP is still phishable



- A fake site can ask for the code and **replay it** within 30s
- It stops password reuse, not a live phishing proxy

2

Recovery codes



- A lost phone must not mean a lost account
- One-time codes, **hashed** like passwords

3

Never reuse a code



- Record the window that was used
- Or one intercepted code works **twice**

Key

- ✓ A **shared secret** plus the clock — nothing travels between the two devices

Takeaways

- ✓ Rank: passkey > TOTP > push > SMS. SMS is the weakest, not "good enough"
- ✓ Burn used windows, hash recovery codes



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

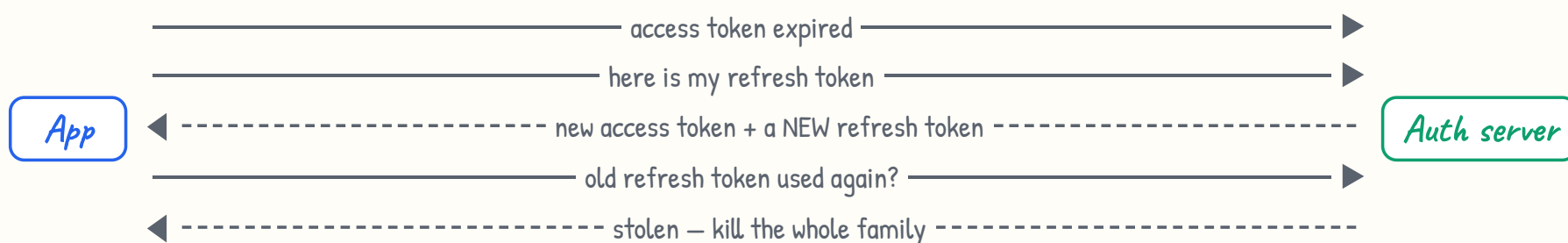
How login actually works — and where it actually breaks

9

Refresh tokens and rotation

How short expiry stops annoying the user

Rotation and reuse detection



rotation: each refresh is single-use, so a replay is proof of theft

1

The split



- Access token: **minutes**, sent on every call
- Refresh token: **weeks**, sent only to the auth server
- Small blast radius where exposure is high

2

Rotate every time



- Each refresh returns a **brand-new** refresh token
- The old one is immediately invalid

3

Reuse = theft



- A used refresh token appearing again means **two holders**
- Revoke the entire token family and force a re-login

4

Store it properly



- An **HttpOnly** cookie, or the device keychain
- Never **localStorage** — that is XSS-readable

Key

Takeaways

- ✓ Short access token + long refresh token = security **and** no constant re-login
- ✓ Rotate on every use; a reused refresh token is **proof of compromise**
- ✓ That detection is the reason rotation exists — **not the new token itself**



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works – and where it actually breaks

10

Service to service – API keys and mTLS

No human, no browser, no login screen

1

API keys



- One long secret string identifying a **caller**, not a user
- Simple; store **hashed**, exactly like a password
- Scope and rate-limit per key, and make rotation routine

2

Why keys leak



- Committed to git, pasted in logs, shipped in mobile apps
- A key in client code is **public** the moment it ships
- Scan your repos – this is the most common real leak

3

mTLS



- Normal TLS: only the **server** proves who it is
- Mutual TLS: the **client** presents a certificate too
- Identity at the transport layer, before any request is read

4

Where mTLS fits



- Inside your own network – service meshes do it by default
- Certificates are short-lived and rotated automatically
- Too heavy for public third-party APIs

5

Signed requests

- The caller signs method, path, body and a **timestamp**
- Replay is blocked by the timestamp and a nonce
- What AWS and payment webhooks use

6

Least privilege



- One credential per service, not one shared everywhere
- Then a leak is **contained** and revocable in isolation

Key

- ✓ API keys identify a **caller**; hash them at rest and rotate on a schedule

Takeaways

- ✓ mTLS moves identity down to the connection – good **inside** your own mesh
- ✓ Never ship a secret in client code, and give **every service its own credential**



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks

11

Interview questions

The ones that actually get asked

1

Why not SHA-256 for passwords?



- "Because it's fast — billions of guesses a second.
- Passwords need a deliberately slow hash with a work factor:
- bcrypt, scrypt or Argon2, plus a unique salt per user."

2

Sessions or JWT?



- "Sessions can be revoked instantly; a JWT is valid until it expires.
- JWT avoids a lookup and travels between services.
- One app → sessions. Many verifiers → short JWT plus refresh."

3

Is a JWT encrypted?



- "No — signed. The payload is base64, readable by anyone holding it.
- The signature only proves it wasn't altered.
- So claims yes, secrets never."

4

Is OAuth authentication?



- "No — it's delegated authorization: may this app do X for this user.
- Identity comes from OpenID Connect on top, via the ID token.
- Treating an access token as proof of identity is the classic bug."

5

Why rotate refresh tokens?



- "So reuse becomes detectable. Each refresh is single-use;
- if an old one appears again, two parties hold it —
- revoke the whole family and force a re-login."

6

401 or 403?



- "401 — I don't know who you are; authenticate.
- 403 — I know, and you still may not.
- Missing the 403 check is how one user reads another's data."



Auth & Security Complete Handbook

Passwords, sessions, JWT, OAuth, SSO, 2FA, mTLS

How login actually works — and where it actually breaks

That's auth, end to end.

What you just went through

- 1 AuthN vs AuthZ
- 2 Storing passwords
- 3 Sessions — the server remembers
- 4 JWT — signed, not encrypted
- 5 Sessions vs JWT
- 6 OAuth 2.0 — not a login system
- 7 OpenID Connect and SSO
- 8 2FA and TOTP
- 9 Refresh tokens and rotation
- 10 Service to service — API keys and mTLS
- 11 Interview questions



Save this — you will come back to it



Share it with whoever owns your login code



Comment **AUTH** for the complete PDF in your DM

Written from 10 years of backend engineering — auth, payments and production systems.

@vikstack